REDIS ROADMAP · PHASE 3 OF 7

SWE Redis
End-to-End Handbook

Session management, rate limiting, message queues and Pub/Sub — the four patterns that take you from "I know `SET` and `GET`" to shipping Redis in production.

2 weeks | Sessions | Rate Limiting | Queues | Pub/Sub | Project 2: Auth Service

Prerequisites: Phase 1 (caching fundamentals) and Phase 2 (Redis data structures & commands) complete.
Companion code: `phase3-redis-swe/`

CONTENTS

What you will learn

Phase 1 and 2 taught you what Redis **is**. Phase 3 teaches you what engineers actually **build** with it. Everything here is state that must be shared across processes, must be fast, and is allowed to expire — the exact shape of problem Redis was designed for.

01 The mental model: what Redis is doing in your backend

02 Session management — HASH + ZSET, sliding TTL, instant revocation

03 Rate limiting — fixed window, sliding window, token bucket (atomic Lua)

04 Message queues — reliable LIST queue, retries, DLQ, Streams

05 Pub/Sub — fan-out, and the message loss you must design for

06 Project 2: Authentication Service (all four, wired together)

07 Production discipline: pipelines, Lua, SCAN, memory, monitoring

08 Command cheat sheet + 14-day plan + self-assessment

HOW TO USE THIS HANDBOOK

Read a module, then open the matching file in the companion repo and read the comments. Then do the exercises at the end of the module **before** moving on. You cannot learn distributed-state behaviour by reading — you learn it by breaking it, e.g. by `kill -9`-ing a worker and watching what Redis still holds.

CHAPTER 01

The mental model

Before any code: there is one question that tells you whether a problem belongs in Redis.

> **THE QUESTION**
>
> **"Is this state shared between processes, hot, and allowed to disappear?"** If yes → Redis. If it must never disappear → your database. If only one process needs it → a local variable.

Why every Phase 3 topic is the same problem

You run four API pods behind a load balancer. Each one is a separate process with its own memory. Now look at what breaks:

Feature	What breaks with in-process memory	Redis structure
Sessions	Login hits pod 1, next request hits pod 3 → "not logged in". Sticky sessions are a hack that dies with the pod.	`HASH` + `ZSET`
Rate limiting	"100/min" silently becomes 400/min with 4 pods, and resets on every deploy.	`STRING` / `ZSET` / `HASH`
Background jobs	An in-memory list of pending work is lost on restart, and cannot be picked up by other machines.	`LIST` / `STREAM`
Live events	Pod 2 has no way to tell pods 1, 3, 4 that something happened.	`PUB/SUB`

One shared, in-memory, TTL-aware store solves all four. That is why Redis is in almost every backend architecture diagram you will ever see.

The single-threaded superpower

Redis executes commands **one at a time** on a single thread. Beginners hear "single threaded" and think "slow". It is the opposite of a problem:

- Everything is in RAM, so a command takes microseconds — 100k+ ops/sec on one core.
- No locks, no mutexes, no race conditions *inside* a command. `INCR` is atomic for free. In Postgres you would need a transaction and a row lock.
- It also means **any slow command blocks everyone**. One `KEYS *` on a million keys, and your whole API stalls. Fear slow commands, not concurrency.

> **THE RULE THIS IMPLIES**
>
> Keep every operation O(1) or O(log N) on *small* data. Never `KEYS`, never `FLUSHALL` in production, no unbounded `LRANGE 0 -1`, no long Lua loops.

Atomicity: three tools, three purposes

Tool	Gives you	Use when
`PIPELINE`	One network round trip for N commands. **Not** atomic.	You have many independent commands and want to kill latency (N×RTT → 1×RTT).
`MULTI/EXEC`	All commands run with nothing in between.	You need several writes to land together, and no command depends on the result of another.
`EVAL` (Lua)	Atomicity *plus* conditional logic — read, decide, write.	"Check the counter, then increment if under the limit." This is why every rate limiter in this handbook is Lua.

You cannot do `if` logic with `MULTI/EXEC` because you do not see any reply until `EXEC` runs. That gap is exactly where the race condition lives — and Lua closes it.

Key naming is your schema

Redis has no tables. Prefixes are the only structure you get, so the convention *is* the design. Every project in this phase uses:

```
<domain>:<entity>:<id>[:<sub>]

sess:9f3a...              # one login session      (HASH)
user:sessions:c81f...     # that user's devices    (ZSET)
rl:slide:login:ip:1.2.3.4 # rate limit window      (ZSET)
q:emails                  # job queue              (LIST)
q:emails:dead             # dead letter queue      (LIST)
events:sessions           # pub/sub channel
```

A good scheme buys you three things: you can `SCAN` one feature's keys, you can attribute memory to a feature, and you can apply a per-feature TTL policy. Ad-hoc key names are technical debt that becomes unfixable once the keys are live.

> **TTL IS NOT OPTIONAL**
>
> Any key created per user, per request or per login **must** have an `EXPIRE`. A session key without a TTL is a memory leak that grows with every login for the entire life of your service. Redis is a store where *forgetting is a feature* — use it.

CHAPTER 02 · MODULE 1

Session management

HTTP is stateless: every request arrives as a stranger. A session is how the server recognises “this is the same person who logged in 10 minutes ago”.

Two designs, one real trade-off

	Stateless JWT	Server-side session (Redis)
Where truth lives	Inside the token the client holds	In Redis, keyed by an opaque random ID
Lookup cost	Zero (verify a signature)	One Redis round trip (~0.3 ms)
Logout	**Impossible** before expiry, unless you add a denylist — which is a session store, so you have re-invented this column	`DEL` → revoked instantly
Permission change	Delayed until the token expires	Effective on the next request
List / kill devices	Not possible	Built in (see the ZSET index)
Payload size	Grows the header on every request	32-byte ID in a cookie

The JWT “advantage” is that it needs no store. Redis makes that advantage almost worthless: a sub-millisecond shared lookup buys you real revocation. That is why most consumer products use server-side sessions (or short-lived JWTs plus a Redis refresh-token store).

The data model

```
Client cookie:  sid=z15zDaQW9cHZ...        (opaque, 256 bits of entropy)
      |
      v
sess:{sid}                    HASH        TTL 30m, refreshed on activity
+---------------------------------------+
| user_id      c81f4e...                |
| email        dev@example.com          |
| ip           203.0.113.9              |
| user_agent   Mozilla/5.0 ...          |
| created_at   1785428890               |   <-- absolute cap is checked against this
| last_seen_at 1785429312               |
| csrf         3J5kNy746C1y...          |
+---------------------------------------+

user:sessions:{user_id}       ZSET        member = sid, score = expires_at
+---------------------------------------+
| sid_laptop   1785430690               |
| sid_phone    1785430712               |   <-- "where am I logged in?"
| sid_tablet   1785430880               |   <-- prune with ZREMRANGEBYSCORE
+---------------------------------------+
```

Why a HASH and not one JSON string?

With a HASH you can read or write a single field (`HGET` / `HSET`) without deserialising and rewriting the whole blob. Updating `last_seen_at` on a JSON string is a read-modify-write cycle — extra latency and a lost-update race.

Why a ZSET for the device index, not a SET?

A SET can list devices, but members do not expire individually, so it slowly fills with dead session IDs. Score the ZSET by expiry and you get two things for free: pruning (`ZREMRANGEBYSCORE key 0 now`, O(log N)) and a fair "max 5 devices" policy — evict the *oldest* with `ZRANGE key 0 0`.

Creating a session

app/sessions.py — create()

```
sid = new_session_id()                # secrets.token_urlsafe(32) — CSPRNG, never random.random()
now = int(time.time())

async with self.r.pipeline(transaction=True) as pipe:   # MULTI/EXEC: 1 RTT, all-or-nothing
    pipe.hset(session_key(sid), mapping=fields)
    pipe.expire(session_key(sid), self.ttl)             # self-destruct: no cleanup cron needed
    pipe.zadd(user_sessions_key(user_id), {sid: now + self.ttl})
    pipe.expire(user_sessions_key(user_id), self.abs_max)
    await pipe.execute()
```

Four commands, one round trip, executed atomically. No other client can observe a half-created session.

> **SECURITY: THE SESSION ID IS A BEARER SECRET**
>
> Whoever holds it *is* the user. So: generate it with a CSPRNG (`secrets`, not `random`), give it ≥128 bits of entropy, and never log it or return it in an error message. When you display sessions to the user, show a prefix only (`z15zDaQW...`).

Reading a session: the sliding window

On every authenticated request we refresh the TTL. An active user is never logged out mid-work; an idle one expires 30 minutes after their last click. But activity alone must not grant immortality, so `created_at` is checked against an absolute cap:

app/sessions.py — get()

```
data = await self.r.hgetall(session_key(sid))
if not data:
    return None                      # never existed, or Redis already expired it

if now - int(data["created_at"]) > self.abs_max:
    await self.destroy(sid)          # hard cap: 12h, even for an active session
    return None

if touch:
    async with self.r.pipeline(transaction=True) as pipe:
        pipe.hset(session_key(sid), "last_seen_at", now)
        pipe.expire(session_key(sid), self.ttl)                        # slide the window
        pipe.zadd(user_sessions_key(data["user_id"]), {sid: now + self.ttl})
        await pipe.execute()
```

Policy	Behaviour	Use for
Fixed / absolute TTL	Expires N minutes after login, no matter what	Banking, admin panels, anything high risk
Sliding TTL	Refreshed on every request	Consumer apps — good UX
Sliding + absolute cap	Refreshed, but hard-killed after 12h	**The sane default**, used here

Revocation — the payoff

app/sessions.py — destroy_all()

```
sids = await self.r.zrange(user_sessions_key(user_id), 0, -1)
async with self.r.pipeline(transaction=True) as pipe:
    for sid in sids:
        pipe.delete(session_key(sid))     # every device, instantly
    pipe.delete(user_sessions_key(user_id))
    await pipe.execute()
```

This is what you run after a password change, a suspicious-login alert, or a breach. It is three lines because the data model was chosen correctly. With stateless JWTs it is a project.

Cookie flags that actually matter

Flag	Why
`HttpOnly`	JavaScript cannot read the cookie, so an XSS bug cannot steal the session.
`Secure`	Sent over HTTPS only. Without it, one plain-HTTP request leaks the session on the wire.
`SameSite=Lax`	Not sent on cross-site POSTs → blocks the classic CSRF form attack.
`Max-Age`	Matches the Redis TTL so the browser stops sending a dead ID.

For non-browser clients (mobile, curl, services) accept the same ID as `Authorization: Bearer <sid>`. One store, two transports — see `app/deps.py :: current_session`.

Mistakes to avoid

Mistake	Consequence
No TTL on session keys	Unbounded memory growth; Redis OOMs weeks later, in production, at 3am.
Sequential or predictable session IDs	Session hijacking by guessing.
Storing the whole user profile in the session	Stale data after an update, and a fat hash on every request. Store `user_id`; look the rest up (and cache it separately — Phase 4).
Not rotating the session ID on login	Session fixation: an attacker plants a known ID, then waits for you to authenticate it.
Using a SET for the device index	Fills with expired IDs forever.
Sessions in a Redis with `allkeys-lru` eviction	Random users get logged out under memory pressure. Sessions need `volatile-*` or a dedicated instance.

Interview answers

Where do you store sessions and why not in the app?

In Redis, because horizontally scaled pods do not share memory, and sticky sessions break as soon as a pod restarts or scales down. Redis gives a shared, sub-millisecond, TTL-aware store — and revocation is a single `DEL`.

JWT or sessions?

JWT when you need stateless verification across service boundaries and can accept delayed revocation — keep them very short lived. Sessions when you need instant logout, device management, or server-controlled permissions. The common hybrid: 10-minute access JWT plus a refresh token stored in Redis so refresh can be revoked.

How do you implement “log out of all devices”?

Keep a per-user index of session IDs (ZSET scored by expiry). On logout-all, pipeline a `DEL` for each session key and drop the index. Then publish an event so other pods invalidate any local cache.

What happens if Redis goes down?

Everyone is logged out — which is why sessions run on a replicated instance with Sentinel/Cluster failover and AOF persistence (Phase 7). It is an availability decision: session loss is annoying but not corrupting, so many teams accept it rather than pay for stronger guarantees.

EXERCISES

1. Add `POST /auth/change-password`: verify the old password, rewrite the hash, then `destroy_all()`. Prove every other device gets 401.
2. Set `SESSION_TTL_SECONDS=10`. Watch `redis-cli ttl sess:<sid>` reset each time you call `/auth/me`, and expire when you stop.
3. Set `MAX_SESSIONS_PER_USER=2` and log in three times. Confirm the oldest session is evicted, not the newest.
4. Add an `impersonated_by` field for admin support login, and make `/auth/me` report it.

CHAPTER 03 · MODULE 2

Rate limiting

A rate limiter answers one question — “has this client had its share in this time window?” — and it must answer identically on every pod, under concurrency, in under a millisecond.

Why it must live in Redis

- **Correctness across pods.** Four pods with local counters turn 100/min into 400/min, and a deploy resets everyone’s budget.
- **Speed.** It runs on every request, so it must not touch a disk-backed DB.
- **Self-cleaning.** Limiter keys are per-client and short lived: exactly what `EXPIRE` is for.

> **THE RACE CONDITION YOU MUST UNDERSTAND**
>
> `GET` → “99, under the limit” → `INCR`. Two concurrent requests both read 99 and both are allowed: the limit is breached. A lock is the wrong fix (slow, deadlock-prone). The right fix is to make check-and-increment a **single atomic operation** — a Lua script, which Redis runs with nothing interleaved.

Algorithm 1 — Fixed window

The window id is part of the key, so a new minute is a brand-new key that dies on its own.

app/rate_limit.py — FIXED_WINDOW_LUA

```
local count = redis.call('INCR', KEYS[1])
if count == 1 then
  -- First hit in this window: attach the TTL. Doing INCR+PEXPIRE inside Lua
  -- guarantees we never create an immortal key (which happens if the client
  -- dies between the two commands).
  redis.call('PEXPIRE', KEYS[1], ARGV[1])
end
return { count, redis.call('PTTL', KEYS[1]) }
```

```
key: rl:fixed:ip:1.2.3.4:29757148     (window id = unix_time / 60)

  |<---- 11:59 ---->|<---- 12:00 ---->|
  |            100  |  100            |      limit = 100/min
                ^^^^^^^
                200 requests inside ~1 second  <-- the boundary burst flaw
```

Cheapest possible limiter (one integer per client) — but a client can spend its whole budget at the end of one window and again at the start of the next.

Algorithm 2 — Sliding window log

Store a timestamp per request in a ZSET, drop everything older than the window, then count. Exact, with no boundary burst.

app/rate_limit.py — SLIDING_WINDOW_LUA

```
local now, window, limit, member = tonumber(ARGV[1]), tonumber(ARGV[2]), tonumber(ARGV[3]),
ARGV[4]

-- Step 1: forget everything older than the window. This is the "sliding" part.
redis.call('ZREMRANGEBYSCORE', KEYS[1], 0, now - window)

local used = redis.call('ZCARD', KEYS[1])
if used < limit then
  -- member must be UNIQUE, otherwise ZADD just updates an existing score
  -- and you silently under-count.
  redis.call('ZADD', KEYS[1], now, member)
  redis.call('PEXPIRE', KEYS[1], window)
  return { 1, limit - used - 1, 0 }
end

-- Denied: tell the client exactly when the oldest request leaves the window.
local oldest = redis.call('ZRANGE', KEYS[1], 0, 0, 'WITHSCORES')
return { 0, 0, (tonumber(oldest[2]) + window) - now }
```

```
rl:slide:login:email:dev@example.com    ZSET, limit 5 per 60s

  now-60s                                          now
     |-------------------------------------------------|
        x    x       x        x     x                      used = 5  -> DENY
        ^oldest: retry_after = (oldest + 60s) - now = 12s
```

Memory cost = limit × active clients, so use it where the limit is small: login, OTP, password reset, signup.

Algorithm 3 — Token bucket

A bucket holds *capacity* tokens and refills at *rate* per second. Each request costs tokens. Bursts are allowed up to the capacity, then throughput settles to the rate. This is what public APIs and LLM gateways use, because real clients are bursty.

app/rate_limit.py — TOKEN_BUCKET_LUA

```
local state  = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens, ts = tonumber(state[1]), tonumber(state[2])
if tokens == nil then tokens, ts = capacity, now end   -- new client starts full

-- LAZY REFILL: no background job tops up every bucket. We compute how many
-- tokens *would* have accrued since the last request. O(1), idle clients free.
local elapsed = math.max(0, now - ts) / 1000.0
tokens = math.min(capacity, tokens + elapsed * rate)

local allowed, retry = 0, 0
if tokens >= cost then tokens = tokens - cost; allowed = 1
else retry = math.ceil(((cost - tokens) / rate) * 1000) end

redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', now)
-- TTL = time for an empty bucket to fill completely: after that the stored
-- state is identical to "new client", so keeping it only wastes memory.
redis.call('PEXPIRE', KEYS[1], math.ceil((capacity / rate) * 1000) + 1000)
return { allowed, math.floor(tokens * 1000), retry }
```

```
capacity 20, refill 2/sec, cost 5 per call

  tokens 20 |####################|  call -> 15
         15 |###############     |  call -> 10
         10 |##########          |  call -> 5
          5 |#####               |  call -> 0
          0 |                    |  call -> 429, Retry-After: 3s
                                    (+2 tokens every second)
```

Choosing

Algorithm	Memory/client	Accuracy	Use for
Fixed window	~8 bytes	Boundary burst	Coarse abuse protection, huge traffic, cheapest option
Sliding window log	limit × ~40 bytes	Exact	Small strict limits: login, OTP, signup, password reset
Token bucket	~80 bytes	Smooth + burst	General API quotas, expensive endpoints, per-plan tiers
Leaky bucket	~80 bytes	Perfectly smooth	When the downstream cannot absorb bursts at all (SMS, payment gateways)

Two-dimensional login limiting

A single dimension is always bypassable. Real login endpoints limit both:

Dimension	Stops	Budget used here
`login:ip:{ip}`	One host brute-forcing many accounts	20 / 60s
`login:email:{email}`	A botnet (many IPs) brute-forcing one account	5 / 60s

app/main.py — login()

```python
ip_check    = await rl.sliding_window(f"login:ip:{ip}", limit * 4, window)
email_check = await rl.sliding_window(f"login:email:{email}", limit, window)
# ... both must pass ...
user = await users.authenticate(email, password)
if user is None:
    raise HTTPException(401, "Invalid email or password")   # never say WHICH is wrong

# SUCCESS -> refund this account's failed-attempt budget, so a legitimate user
# logging in repeatedly is never locked out by their own past typos.
await r.delete(f"rl:slide:login:email:{email}")
```

DO NOT LOCK THE ACCOUNT

"5 failures = account locked for 1 hour" is a denial-of-service weapon: I can lock any user I know the email of. Rate limit the *attempt*, not the account, and add CAPTCHA or step-up verification after N failures.

Respond like a good citizen

```
HTTP/1.1 429 Too Many Requests
Retry-After: 12                 # seconds — well-behaved clients back off
X-RateLimit-Limit: 5
X-RateLimit-Remaining: 0
```

Without `Retry-After`, clients retry in a tight loop and turn your limiter into an amplifier. Also: **never** rate limit by IP alone for logged-in users — corporate NAT and mobile carriers put thousands of people behind one address. Prefer user ID when you have one (`rate_limit(..., per="user")` in `app/deps.py`).

Mistakes to avoid

Mistake	Consequence
GET-then-INCR in application code	Race condition under load; the limit leaks. Use Lua.
`INCR` without `EXPIRE` in the same script	Immortal keys; the client is banned forever.
Concatenating the key into the Lua body	Breaks Redis Cluster routing (keys must be in `KEYS[]`) and invites injection.
Trusting `X-Forwarded-For` blindly	Clients forge it and get unlimited budgets. Only trust the hop your own LB writes.
Limiting only by IP	One NAT'd office building shares one budget.
Failing closed when Redis is down	A cache outage becomes a total outage. Decide deliberately: fail-open (allow, log, alert) is right for most product endpoints; fail-closed for payments and OTP.

Interview answers

Design a rate limiter for 10k requests/sec across 20 servers.

Token bucket in Redis, one Lua script per request, keyed by API key. One Redis node handles that comfortably (~100k ops/sec); shard by key hash if it does not. Add a small per-pod local pre-filter for obvious abusers to cut Redis traffic, and return `429 + Retry-After`. If Redis is unreachable, fail open and alert.

Why Lua instead of MULTI/EXEC?

Because the decision depends on the current value. Inside `MULTI` you get no replies until `EXEC`, so you cannot branch. Lua executes atomically *and* can branch. (`WATCH`-based optimistic locking also works but retries under contention, which is exactly when a limiter is busiest.)

Fixed window's flaw?

Boundary burst: up to 2× the limit across a window edge. Fix with a sliding window log (exact, more memory) or a sliding-window counter that weights the previous window.

EXERCISES

1. Switch `/api/expensive` to fixed window and *demonstrate* the boundary burst: 2× the limit inside one second.
2. Add per-plan limits (`free 10/min` , `pro 200/min`) read from the user hash.
3. Make `cost` proportional to work — e.g. cost = number of LLM tokens requested. This is exactly how paid AI APIs meter you.
4. Return `X-RateLimit-*` headers on *successful* responses too.

CHAPTER 04 · MODULE 3

Message queues

The single most common backend refactor in the industry: take the slow, failure-prone work out of the request path and hand it to a worker.

```
BEFORE                                  AFTER
POST /register                          POST /register
  write user      5ms                     write user     5ms
  send email    800ms  <-- user waits     LPUSH job      1ms
  201                                     201            6ms
                                               |
  mail provider down => signup 500          worker (separate process, retries)
                                               |
                                            email sent, or dead-lettered
```

Lists as a queue

`LPUSH` on the left, pop from the right → FIFO. Both are O(1) no matter how deep the queue is. The naive version loses jobs:

Pattern	Guarantee	Verdict
`LPUSH` + `BRPOP`	At-most-once. Worker crashes after popping → the job existed only in its RAM → gone.	Fine for metrics, never for emails or payments
`LPUSH` + `BLMOVE` + `LREM`	At-least-once. The job is always in Redis: either queued, in a processing list, or acknowledged.	**The reliable-queue pattern** — what we build

```
  producer                    Redis                          worker
     |                                                         |
  LPUSH q:emails  ------>  q:emails  [j5][j4][j3]              |
                              |                                 |
                    BLMOVE RIGHT->LEFT (atomic)  <--------------+
                              v
                q:emails:processing:{worker_id}  [j3]     job is IN FLIGHT
                              |
        success -> LREM processing 1 j3                 <-- this is the ACK
        failure -> ZADD q:emails:delayed  now+backoff    <-- retry later
        dead    -> LPUSH q:emails:dead                   <-- human inspects
```

app/queue/worker.py — the consume loop

```
while self.running:
    # timeout=2 -> returns None every 2s so we can re-check `running` and shut
    # down cleanly. NEVER use timeout=0 (block forever) in a process you deploy.
    body = await self.r.blmove(queue_key(q), processing_key, 2, "RIGHT", "LEFT")
    if body is None:
        continue
    await self._execute(body, processing_key)
```

WHY THE WORKER BUILDS ITS OWN REDIS CLIENT

A blocking command holds a connection for its entire timeout. A worker blocking 2s at a time would permanently occupy a connection from the web app's pool. Blocking consumers and Pub/Sub subscribers always get their own pool — see `new_client()` in `app/redis_client.py`.

Retries with exponential backoff

app/queue/worker.py — _handle_failure()

```
job["attempts"] += 1
if job["attempts"] < MAX_ATTEMPTS:
    backoff = 2 ** job["attempts"]                      # 2s, 4s, 8s ...
    async with self.r.pipeline(transaction=True) as pipe:
        pipe.zadd(delayed_key(q), {json.dumps(job): time.time() + backoff})
        pipe.lrem(processing, 1, body)                   # ACK the old copy
        await pipe.execute()
else:
    async with self.r.pipeline(transaction=True) as pipe:
        pipe.lpush(dead_key(q), json.dumps(job))         # dead letter queue
        pipe.lrem(processing, 1, body)
        await pipe.execute()
```

Backoff matters: retrying a failing downstream immediately turns your worker fleet into a DDoS against it. The dead letter queue matters because one poison message must never block the queue forever — park it where a human can look.

The delayed set and its scheduler

Delayed and retry jobs live in a ZSET scored by their run time. A loop promotes the due ones — atomically, or two workers would both promote the same job:

```
local due = redis.call('ZRANGEBYSCORE', KEYS[1], 0, ARGV[1], 'LIMIT', 0, 100)
for _, job in ipairs(due) do
  redis.call('ZREM', KEYS[1], job)     -- claim it
  redis.call('LPUSH', KEYS[2], job)    -- and queue it
end
return #due
```

The same ZSET pattern gives you scheduled jobs for free: `enqueue(..., delay_seconds=3600)` = "send this reminder in an hour".

Recovering jobs from dead workers

If a worker is `kill -9`'d, its processing list still holds the job. A recovery loop requeues anything in flight longer than the visibility timeout:

```
async for key in self.r.scan_iter(match=f"q:{q}:processing:*", count=100):
    for body in await self.r.lrange(key, 0, -1):
        if time.time() - job["enqueued_at"] > visibility_timeout:
            async with self.r.pipeline(transaction=True) as pipe:
                pipe.lpush(queue_key(q), body)
                pipe.lrem(key, 1, body)
                await pipe.execute()
```

SCAN, NEVER KEYS

`KEYS pattern` walks the entire keyspace in one blocking call — on a single-threaded server that means every other client waits. `SCAN` returns a cursor in small batches. This is one of the most common production incidents caused by Redis beginners.

AT-LEAST-ONCE MEANS HANDLERS MUST BE IDEMPOTENT

A job can run twice: the worker did the work, then died before the ACK. Guard it — `SET job:done:{id} 1 NX EX 86400` before acting, and skip if you did not win the key. “Exactly once” does not exist across a network; “at-least-once + idempotent” is the real engineering answer, and interviewers listen for exactly that sentence.

Job envelope design

```
{
  "id": "5c7115ca...",          // for idempotency and tracing
  "task": "welcome_email",      // which handler runs it
  "payload": {"email": "..."},  // arguments only — no objects, no DB rows
  "attempts": 0,                // retry bookkeeping
  "enqueued_at": 1785428939.26  // latency metrics + stuck detection
}
```

- **JSON, never pickle.** The queue crosses process and language boundaries, and unpickling attacker-influenced bytes is remote code execution.
- **Pass IDs, not data.** Push `user_id`, let the worker load the row. A snapshot embedded in the job is stale the moment it is queued.
- **Keep payloads small.** Big blobs belong in S3; queue the key.

The upgrade: Redis Streams

Everything above is application logic you had to write. Streams put it inside Redis:

Command	What it does for you
`XADD`	Append to an immutable log; the ID is a timestamp+sequence
`XGROUP CREATE`	Define a consumer group = "the workers of this queue"
`XREADGROUP`	Read new entries *and* record them as pending (the PEL) — delivery tracking for free
`XACK`	Mark done; removes it from the pending list
`XAUTOCLAIM`	Steal entries idle longer than N ms — crash recovery for free
`XTRIM MAXLEN ~ 10000`	Cap memory. **Forgetting this is the #1 Streams production mistake.**

Option	Delivery	History	Pick it when
Pub/Sub	At-most-once	None	Ephemeral notifications only
LIST	At-least-once (with ACK logic you write)	Until consumed	Simple FIFO jobs; you want full control
STREAM	At-least-once, built in	Retained + replayable by many groups	**Default for new work**
Kafka	At-least-once / effectively-once	Days–weeks, huge scale	You have outgrown Redis: multi-DC, TB of retention

Run `python -m app.queue.streams_demo` to see consumer groups splitting work, `XACK`, and `XAUTOCLAIM` in about 40 lines.

Operate it

- **Graph queue depth** (`LLEN`). Rising depth = consumers are losing. It is your earliest warning signal.
- **Alert on the DLQ being non-empty.** A silent DLQ is lost customer email.
- **Handle SIGTERM.** Stop taking new jobs, finish the current one, exit — otherwise every deploy leaves half-done work.
- **Separate queues by priority** (`q:emails:high`, `q:emails:low`) and read the high one first. One slow bulk job must not delay password resets.
- **Never run a job queue on a Redis with `allkeys-lru`** — eviction would silently delete jobs. Use `noeviction`, or a separate instance from your cache.

Interview answers

Build a job queue on Redis. How do you not lose jobs?

Reliable queue: `BLMOVE` from the queue into a per-worker processing list, so the job is atomically transferred and always visible in Redis. ACK by `LREM`-ing it after success. Retries go to a delayed ZSET with exponential backoff; after N attempts, to a dead letter list. A recovery loop requeues entries idle past the visibility timeout. Handlers are idempotent because delivery is at-least-once.

Redis queue vs Kafka vs SQS?

Redis: lowest latency, you already run it, no retention beyond memory — perfect up to millions of jobs/day. Kafka: durable multi-consumer log with long retention, for event-sourcing and analytics scale. SQS: managed, no ops, higher latency. Choose by retention and ops budget, not by fashion.

Ordering guarantees?

A single list with one consumer is FIFO. Add consumers and per-key ordering is gone. If order matters, shard by entity (all jobs for user X on one queue/partition) so ordering is preserved where it is semantically needed.

> EXERCISES
>
> 1. `kill -9` the worker mid-job; watch `requeue_stuck_jobs()` rescue it. Then set the visibility timeout to 1s and observe *duplicate* execution — that is why idempotency is non-negotiable.
> 2. Add the `SET job:done:{id} 1 NX` guard and prove duplicates become no-ops.
> 3. Run two workers; confirm jobs are split, not duplicated.
> 4. Add `q:emails:high` and consume it first.
> 5. Write a `/admin/dlq/replay` endpoint that moves dead letters back.

CHAPTER 05 · MODULE 4

Pub/Sub

One publisher, many listeners, zero memory. Powerful and simple — as long as you fully accept its one hard limitation.

```
pod #2:  PUBLISH events:sessions '{"type":"logout_all","user_id":"c81f"}'
                        |
                      Redis
            +-------------+-------------+
            v             v             v
      pod #1        pod #3        pod #4        (currently SUBSCRIBEd)
    drop local    drop local    drop local
    cached copy   cached copy   cached copy

      pod #5 was restarting  ->  NEVER receives this message. Ever.
```

THE ONE THING TO REMEMBER

Pub/Sub is **fire-and-forget, at-most-once**. No storage, no ACK, no retry, no replay. A subscriber that is offline at publish time misses the message permanently. `PUBLISH` returns the number of receivers — if that is 0, the message is simply gone.

So use it only where loss is survivable

Good fit	Why loss is acceptable
Cache / local-state invalidation	The local copy has a short TTL anyway, so a missed event self-heals in seconds
WebSocket fan-out between pods	Only the pod holding the socket cares, and clients re-sync on reconnect
Live dashboards, presence, “user is typing”	The next tick overwrites it
Config hot-reload signal	Falls back to a periodic poll

Wrong fit	Use instead
Sending emails, charging cards, provisioning	LIST or STREAM queue
Anything you must audit or replay	STREAM (retained) or Kafka
Anything where duplicates or gaps corrupt state	STREAM + idempotent consumer

How we use it in Project 2

Each API pod may keep a tiny in-process session cache to save round trips. When a user clicks “log out everywhere”, the pod handling it deletes the Redis keys — but other pods still hold stale copies in local memory. One `PUBLISH` tells them to drop it

immediately:

app/pubsub/events.py

```
async def publish_session_event(redis, event_type, **data) -> int:
    message = json.dumps({"type": event_type, "at": time.time(), **data})
    # Returns the number of subscribers that received it. LOG THIS: a persistent
    # 0 means your consumers are dead and nobody noticed — the classic silent
    # Pub/Sub failure.
    return await redis.publish(CHANNEL_SESSIONS, message)
```

app/pubsub/events.py — subscriber

```
pubsub = r.pubsub(ignore_subscribe_messages=True)
await pubsub.subscribe(CHANNEL_SESSIONS)

while True:
    # get_message() also returns subscribe confirmations and returns None while
    # nothing is buffered -> always poll in a loop, never assume one call is enough.
    msg = await pubsub.get_message(timeout=1.0)
    if msg is None:
        continue
    await handle_event(json.loads(msg["data"]))
```

CONNECTION RULE

A subscribed connection enters a special mode where only `SUBSCRIBE/UNSUBSCRIBE/PING/QUIT` are legal. A subscriber therefore **must** have its own connection — never the shared request pool. And it must reconnect+resubscribe on disconnect, or it goes deaf silently.

Details worth knowing

- **Channels are not declared.** `PUBLISH` to a channel nobody listens to is a no-op — no error, no queue, nothing created.
- **Patterns:** `PSUBSCRIBE 'events:*'` works, but Redis matches the pattern on every publish. Keep the number of patterns small.
- **Slow subscribers get disconnected.** If a client cannot keep up, its output buffer hits `client-output-buffer-limit` and Redis drops it. Handlers must be fast: parse the event, hand the work to a queue, return.
- **Cluster:** classic Pub/Sub is broadcast to all nodes; `SSUBSCRIBE` (sharded Pub/Sub, Redis 7+) keeps traffic on one shard and scales far better.
- **Keyspace notifications** (`notify-keyspace-events Ex`) publish an event when a key expires. Tempting for “session expired” hooks — but delivery is Pub/Sub, so it is best-effort. Never build billing on it.

Interview answers

Pub/Sub vs Streams?

Pub/Sub is at-most-once with no storage: subscribers present at publish time get it, everyone else misses it. Streams persist entries, support consumer groups with ACKs and claim-on-crash, and let several independent groups replay the same history. Notifications → Pub/Sub. Work → Streams.

How do you invalidate caches across 20 servers?

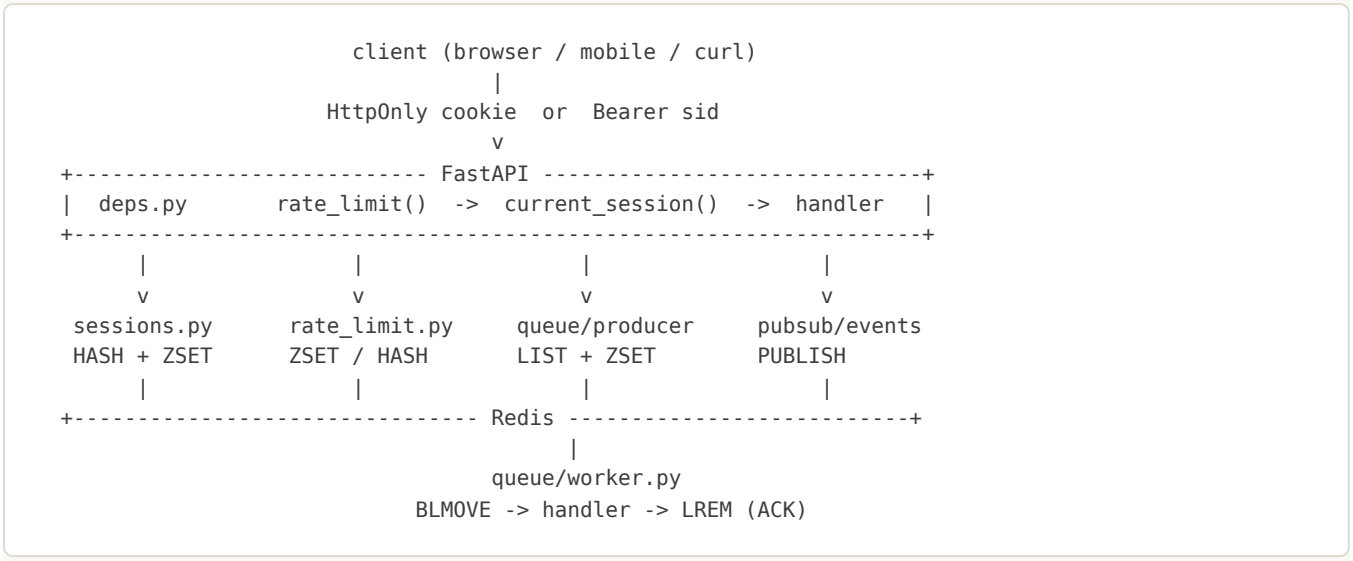
Publish the invalidation key on a channel; every pod subscribes and evicts its local entry. Keep short local TTLs so a missed message self-heals, and version the cache key so a stale read is impossible for correctness-critical data.

> EXERCISE
>
> Stop the subscriber, trigger `/auth/logout-all`, restart it — the event is gone forever. Now redo the same flow with a Stream and see that the entry is still waiting. That contrast *is* the lesson of this chapter.

CHAPTER 06

Project 2 — Authentication Service

All four modules in one service. This is the artefact you put on GitHub and talk about in interviews.



Endpoints

Endpoint	Redis mechanics on display
`POST /auth/register`	`SET NX` as an atomic email claim; `LPUSH` the welcome-email job; fixed IP limit 3/hour
`POST /auth/login`	Two-dimensional sliding-window limit; `MULTI` session create; cookie; `PUBLISH` login event
`GET /auth/me`	`HGETALL` + sliding `EXPIRE` + absolute-cap check
`GET /auth/sessions`	`ZREMRANGEBYSCORE` prune, `ZRANGE`, then pipelined `HGETALL` burst
`POST /auth/logout`	`DEL` + `ZREM` + `PUBLISH`
`POST /auth/logout-all`	Pipelined mass revocation + fan-out event
`POST /auth/forgot`	`SETEX` one-time token (15 min) + queued email; identical response whether or not the account exists
`GET /api/expensive`	Token bucket, cost 5 per call, per user
`GET /health`	`PING` — a health check that does not test its dependency is worse than none
`GET /admin/queue`	`LLEN` / `ZCARD` depth metrics

The login request, step by step

```
1  POST /auth/login {email, password}
2  ZSET  rl:slide:login:ip:1.2.3.4        -> 20/60s   pass?  else 429 + Retry-After
3  ZSET  rl:slide:login:email:dev@..      ->  5/60s   pass?  else 429
4  GET   user:byemail:dev@example.com     -> user_id  (constant-time dummy hash if missing)
5  HGETALL user:{user_id}                 -> verify PBKDF2 hash
6  DEL   rl:slide:login:email:dev@..      -> refund the budget on success
7  MULTI  HSET sess:{sid} ... ; EXPIRE 1800 ; ZADD user:sessions:{uid} ; EXPIRE  EXEC
8  ZREMRANGEBYSCORE + evict oldest if over max_sessions_per_user
9  Set-Cookie: sid=...; HttpOnly; SameSite=Lax; Secure; Max-Age=1800
10 PUBLISH events:sessions {"type":"login",...}
11 200 {session_id, csrf_token, expires_in}
```

Security decisions embedded in the code

Decision	Attack it defeats
PBKDF2 with 200k iterations + per-user salt	Offline brute force of a stolen hash dump
Dummy hash computed when the email is unknown	User enumeration via response timing
Identical 401 message for bad email and bad password	User enumeration via error text
`/auth/forgot` always returns 200	User enumeration via the reset endpoint
`hmac.compare_digest`	Timing attacks on secret comparison
New session ID issued at login	Session fixation
Rate limit the attempt, never lock the account	Attacker-triggered account lockout DoS
Reset token in Redis with a 15-minute TTL	Stolen-link replay; expiry is automatic

Run it

```
docker compose up -d                    # Redis 7 with AOF, maxmemory-policy noeviction
pip install -r requirements.txt

uvicorn app.main:app --reload           # terminal 1: API   -> /docs
python -m app.queue.worker              # terminal 2: worker
python -m app.pubsub.events             # terminal 3: subscriber

python -m scripts.smoke_test            # narrated end-to-end walkthrough
bash scripts/demo.sh                    # the same, with raw curl
```

Watch it work from redis-cli

```
redis-cli --scan --pattern 'sess:*'           # every live session
redis-cli hgetall sess:<sid>                  # its fields
redis-cli ttl sess:<sid>                      # call /auth/me and watch it reset
redis-cli zrange user:sessions:<uid> 0 -1 WITHSCORES
redis-cli zrange rl:slide:login:email:dev@example.com 0 -1 WITHSCORES
redis-cli llen q:emails ; redis-cli llen q:emails:dead
redis-cli subscribe events:sessions           # see the fan-out live
redis-cli monitor                             # every command (dev only, very loud)
```

EXTEND IT (PORTFOLIO-GRADE ADDITIONS)

Email verification tokens · TOTP two-factor with a Redis replay guard · OAuth state parameter stored with a 5-minute TTL · admin impersonation with an audit trail · per-plan rate limits · distributed lock (`SET NX PX` + Lua-checked release) · Prometheus metrics for queue depth and 429 rate.

CHAPTER 07

Production discipline

These are the habits that separate “it worked on my laptop” from “it survived Black Friday”. Phase 7 goes deep; this is the subset you need the moment code ships.

1. Connections

- One client + pool per process, created at startup, closed at shutdown. Never inside a handler.
- Pool size ≈ your web server’s concurrency, not “as big as possible”. Every pool multiplies by pod count against Redis’ `maxclients`.
- Always set `socket_timeout` and `socket_connect_timeout`. Without them, a network blip hangs requests until the client gives up — which may be never.
- Blocking consumers and Pub/Sub subscribers get their own pool.

2. Latency: round trips are the cost, not Redis

```
# BAD: 100 round trips (~100 x 0.4ms = 40ms of pure waiting)
for sid in sids:
    data = await r.hgetall(f"sess:{sid}")

# GOOD: 1 round trip
async with r.pipeline(transaction=False) as pipe:
    for sid in sids:
        pipe.hgetall(f"sess:{sid}")
    rows = await pipe.execute()
```

An N+1 pattern against Redis is just as damaging as against SQL. Pipeline it, or move the logic into Lua.

3. Commands that will hurt you

Never	Instead	Why
`KEYS pattern`	`SCAN` / `HSCAN` / `SSCAN`	O(N) over the whole keyspace, blocks every client
`FLUSHALL` / `FLUSHDB`	Delete by prefix with SCAN	Instant, unrecoverable data loss
`LRANGE key 0 -1` on a big list	Paged ranges	Serialises megabytes into one reply
`DEL` on a huge collection	`UNLINK`	Frees memory on a background thread
`SMEMBERS` on a huge set	`SSCAN`	Same blocking problem
`MONITOR` in production	`SLOWLOG`, metrics	Mirrors every command; costs real throughput

4. Memory and eviction — the trap in this phase

`maxmemory-policy` is global to an instance. That is fine for a pure cache and dangerous once the same instance also holds sessions and queues:

Policy	Effect	Right for
`allkeys-lru`	Evicts anything, including sessions and queued jobs	Cache-only instances
`volatile-lru`	Evicts only keys that already have a TTL	Mixed workloads (sessions have TTLs; jobs must not)
`noeviction`	Writes fail once memory is full — loud, not silent	Queues, sessions, anything whose loss is a bug

> **THE CLASSIC OUTAGE**
>
> Cache and queue share one Redis with `allkeys-lru`. Traffic spikes, memory fills, Redis evicts "least recently used" keys — including `q:emails`. Jobs vanish with no error anywhere. **Cache and queue belong on separate instances (or at minimum separate instances with different policies — not just different `SELECT` databases).**

5. Failure behaviour: decide it on purpose

Redis is down...	Fail open (allow)	Fail closed (deny)
Rate limiter	Product endpoints — an outage of a protection layer should not be a total outage	Payments, OTP, signup
Sessions	Not an option — no session store means no auth	Return 503 and page someone
Queue	Degrade: do the work inline, or return 202 and reconcile later	Reject the write if the job is essential

Wrap Redis calls in a timeout, catch the error, emit a metric, and take the decision you wrote down — not whatever the stack trace happens to do.

6. Observability

```
redis-cli info memory | grep -E 'used_memory_human|maxmemory_human|evicted'
redis-cli info clients                # connected_clients, blocked_clients
redis-cli info stats | grep -E 'keyspace_hits|keyspace_misses|expired_keys'
redis-cli slowlog get 10              # commands over 10ms — check this weekly
redis-cli --latency                   # live latency
redis-cli --bigkeys                   # find the one collection that will hurt you
redis-cli --memkeys                   # memory per key pattern
redis-cli info commandstats | head    # which command dominates
```

Graph and alert on: hit rate, `used_memory` vs `maxmemory`, `evicted_keys` (should be 0 for non-cache instances), `blocked_clients`, queue depth, DLQ size, 429 rate, and p99 command latency.

7. Habits checklist

- [ ] Every per-user / per-request key has a TTL
- [ ] All keys follow one documented naming convention
- [ ] Check-then-set logic lives in Lua, not in Python
- [ ] Keys are passed as `KEYS[]`, never concatenated into the script body
- [ ] Pipelines used wherever the app loops over keys
- [ ] `SCAN` instead of `KEYS` everywhere
- [ ] Timeouts set on the client; failure mode chosen deliberately
- [ ] Cache and queue are not sharing an eviction policy
- [ ] Queue workers are idempotent and handle SIGTERM
- [ ] Dead letter queue is monitored and alerted on
- [ ] Secrets (session IDs, tokens) never appear in logs

CHAPTER 08

Cheat sheet & study plan

Commands used in this phase

Sessions (HASH + ZSET)	Purpose
`HSET k f v [f v ...]`	Write session fields
`HGETALL k` / `HGET k f`	Read all / one field
`EXPIRE k s` / `TTL k`	Set / inspect the sliding window
`PERSIST k`	Remove a TTL (you almost never want this)
`ZADD k score member`	Index a session under its user
`ZRANGE k 0 -1 [WITHSCORES]`	List devices, oldest first
`ZREMRANGEBYSCORE k 0 now`	Prune expired index entries
`ZCARD k` / `ZREM k m`	Count / remove a device
`DEL k` / `UNLINK k`	Revoke (UNLINK for big keys)

Rate limiting	Purpose
`INCR k` / `INCRBY k n`	Fixed-window counter
`PEXPIRE k ms` / `PTTL k`	Millisecond TTL and remaining time
`SET k v NX EX s`	Atomic claim: locks, one-time tokens, idempotency guards
`SETEX k s v`	Value + TTL in one command
`EVAL script numkeys k... arg...`	Atomic check-and-decide
`SCRIPT LOAD` / `EVALSHA`	Send the body once, then only the SHA
`HMGET` / `HSET`	Token-bucket state

Queues	Purpose
`LPUSH k v` / `RPUSH k v`	Enqueue (left) / enqueue at the tail
`BRPOP k timeout`	Blocking pop — at-most-once, avoid for real work
`BLMOVE src dst RIGHT LEFT t`	Reliable consume (atomic transfer)
`LREM k 1 v`	ACK: remove from the processing list
`LLEN k` / `LRANGE k 0 -1`	Depth / inspect (careful on big lists)
`ZADD delayed score job`	Schedule a retry or a future job
`ZRANGEBYSCORE k 0 now LIMIT 0 100`	Find due jobs
`SCAN cursor MATCH p COUNT n`	Find stuck processing lists

Streams & Pub/Sub	Purpose
`XADD s * f v`	Append an entry
`XGROUP CREATE s g $ MKSTREAM`	Create a consumer group
`XREADGROUP GROUP g c COUNT n BLOCK ms STREAMS s >`	Consume + track as pending
`XACK s g id`	Acknowledge
`XAUTOCLAIM s g c min-idle-time 0`	Recover a dead consumer's entries
`XPENDING s g` / `XINFO GROUPS s`	In-flight and lag inspection
`XTRIM s MAXLEN ~ 10000`	Cap memory (do not skip this)
`PUBLISH ch msg`	Fan-out; returns receiver count
`SUBSCRIBE ch` / `PSUBSCRIBE p`	Listen / pattern listen
`PUBSUB CHANNELS` / `NUMSUB ch`	Who is listening right now

The 14-day plan

Day	Do this
1–2	Read Chapters 1–2. Build the session store from scratch, no copy-paste. Inspect every key in `redis-cli`.
3	Sliding TTL, absolute cap, device limit, `/auth/sessions`. Session exercises 1–4.
4–5	Chapter 3. Implement all three limiters. Write a script that fires 200 concurrent requests and prove the limit holds exactly.
6	Two-dimensional login limiting, 429 + `Retry-After`, per-plan limits.
7	Rest / review. Re-explain sessions and all three limiters out loud, from memory, no notes.
8–9	Chapter 4. Naive `BRPOP` queue first — then `kill -9` a worker and lose a job on purpose. Feel it, then rebuild with `BLMOVE`.
10	Retries, backoff, DLQ, stuck-job recovery, idempotency guard, two workers.
11	Rewrite the queue with Streams + consumer groups. Compare the two implementations line by line.
12	Chapter 5. Pub/Sub invalidation, and prove message loss with a restarted subscriber.
13	Finish Project 2: health check, metrics, graceful shutdown, README, docker compose.
14	Chapter 7 checklist over your own code. Then write a blog post or README section explaining every design decision — teaching is the test.

Self-assessment — can you do these without notes?

- [ ] Explain why sessions cannot live in process memory across 4 pods
- [ ] Justify server-side sessions vs JWT in both directions
- [ ] Draw the HASH + ZSET session model and explain each choice
- [ ] Implement sliding TTL with an absolute cap
- [ ] Implement “log out of all devices” and say why JWT cannot
- [ ] Name the fixed-window flaw and demonstrate it
- [ ] Write the sliding-window ZSET limiter in Lua from memory
- [ ] Explain lazy refill in a token bucket
- [ ] Explain why check-then-set needs Lua and not MULTI/EXEC
- [ ] Explain why `BRPOP` loses jobs and how `BLMOVE` fixes it
- [ ] Explain at-least-once and write an idempotency guard
- [ ] Explain retry backoff and the purpose of a DLQ
- [ ] State the difference between Pub/Sub, Streams and Kafka in one sentence each
- [ ] Say why `KEYS` is dangerous and what to use instead
- [ ] Say what `allkeys-lru` does to a job queue

NEXT: PHASE 4 — SYSTEM DESIGN REDIS

You now hold every SWE pattern. Phase 4 asks what breaks at scale: cache invalidation strategies, the cache stampede (and its fixes — locking, early recompute, jittered TTLs), cache warming, replication, Sentinel and Cluster. Everything you built here becomes the worked example: “what happens to my session store when the master fails over?”

EFFORT SPLIT, RESTATED

Most engineers can recite Redis commands. Far fewer can explain why a limiter needs Lua, why a queue needs an ACK, or what eviction does to a session store. **The commands are the easy 30%. The reasoning in this handbook is the 70% that gets you hired.**